

Programmazione di Dispositivi Mobili



“6 discalculici”

Messa Alessandro 903321

Paschetto Leonardo 894467

Ronchi Matteo 899659

Tentori Francesca 899835

Teruzzi Luca 894437

A.A 2024/2025

SOMMARIO

1	Introduzione.....	3
2	Tecnologie utilizzate	4
3	Design	5
3.1	Progettazione UI	5
4	Architettura	6
4.1	Database	7
4.2	Model	7
4.3	Repository	7
4.4	Service	7
4.5	Source	8
4.6	Offline-First.....	8
5	Funzionalità.....	9
5.1	Tag.....	9
5.2	Battle log.....	10
5.3	Brawler	11
5.4	Stats	12
5.5	BrawlerDetails	13
6	Sviluppi futuri.....	14
6.1	Suggerimenti Brawler per le mappe.....	14
6.2	Espansione a più giochi mobile	14

1 INTRODUZIONE

Gamerise è un'applicazione mobile per Android che permette ai giocatori di *Brawl Stars* di tenere traccia delle proprie statistiche di gioco, consentendo loro di visualizzare tutte le partite giocate, l'elenco dei brawler posseduti (con i relativi gadget e star power) e le statistiche di gioco.

Una volta effettuato l'accesso, inserendo il proprio tag (identificativo del proprio account di Brawl Stars, ottenibile ispezionando il proprio profilo nel gioco) sarà possibile visualizzare:

- Tutte le battaglie disputate, con l'indicazione dei trofei vinti o persi.
- L'elenco dei brawler posseduti con:
 - Numero di *trofei* ottenuti col personaggio
 - Il livello del brawler
 - Le *star power* e i *gadget*
 - Quantità di *monete* e *power point* necessari per arrivare all'ultimo livello, l'undicesimo.
- Statistiche sulle partite effettuate, tra cui grafici con
 - Numero di vittorie su ciascuna mappa
 - Numero di vittorie per modalità

L'applicazione consente il login tramite Google: una volta effettuato il tag salvato nell'app verrà sincronizzato e salvato in cloud. Fatto ciò, dovesse l'utente fare l'accesso da un altro dispositivo non dovrà ricordarsi la propria tag in quanto verrà automaticamente scaricata da Firebase.

2 TECNOLOGIE UTILIZZATE

- **Android studio:** ambiente di sviluppo integrato (IDE) basato su IntelliJ, impiegato per la creazione di applicazioni Android utilizzando il linguaggio Java.
- **GitHub:** piattaforma utilizzata per facilitare la collaborazione tra gli sviluppatori durante l'implementazione dell'applicazione.
Abbiamo utilizzato il branch main come master per le release, per lo sviluppo di ogni nuova feature abbiamo creato un branch (ad esempio, localDataSource o Firebase).
- **Firebase:** piattaforma sviluppata da Google che consente la gestione del database dell'applicazione e supporta le funzionalità di autenticazione degli utenti. In particolare, i servizi utilizzati sono:
 - **Authentication:** database in cui vengono i dati dell'autenticazione con Google.
 - **Real-time database:** database in cui viene salvato il tag preferito dell'utente.
- **API:** API esterna che fornisce i dati di Brawl Stars <https://developer.brawlstars.com/#/>
 - Per utilizzare l'API bisogna registrarsi sul sito e creare un API Key, ma non solo: bisogna anche specificare (a priori) uno o più IP statici da cui le richieste API verranno effettuate. Questa limitazione esiste per evitare attacchi DDoS ai server del gioco, ma è una limitazione enorme per la nostra app perché anche solo in fase di sviluppo, ogni volta che ci si connette ad una rete diversa bisognerebbe generare una nuova API Key. Per aggirare questa limitazione abbiamo creato un Proxy in python che esiste su un server con IP fisso: abbiamo creato una API Key con questo IP come autorizzato, così ora possiamo mandare le richieste al proxy, il quale le autentica e le inoltra ai server di Brawl Stars, e ci ritorna la risposta.

3 DESIGN

3.1 PROGETTAZIONE UI

Per progettare l'interfaccia utente dell'applicazione abbiamo utilizzato uno strumento di progettazione grafica denominato *Figma*. In particolare, è stato impiegato per sviluppare dei prototipi per le schermate, fornendo così una base di riferimento per l'implementazione delle schermate definitive su Android Studio.

Lo stile di alcuni Components è preso da **Material Design 3**. In particolare, lo abbiamo utilizzato per i Button, la Navigation Bar e per alcuni Image Text.

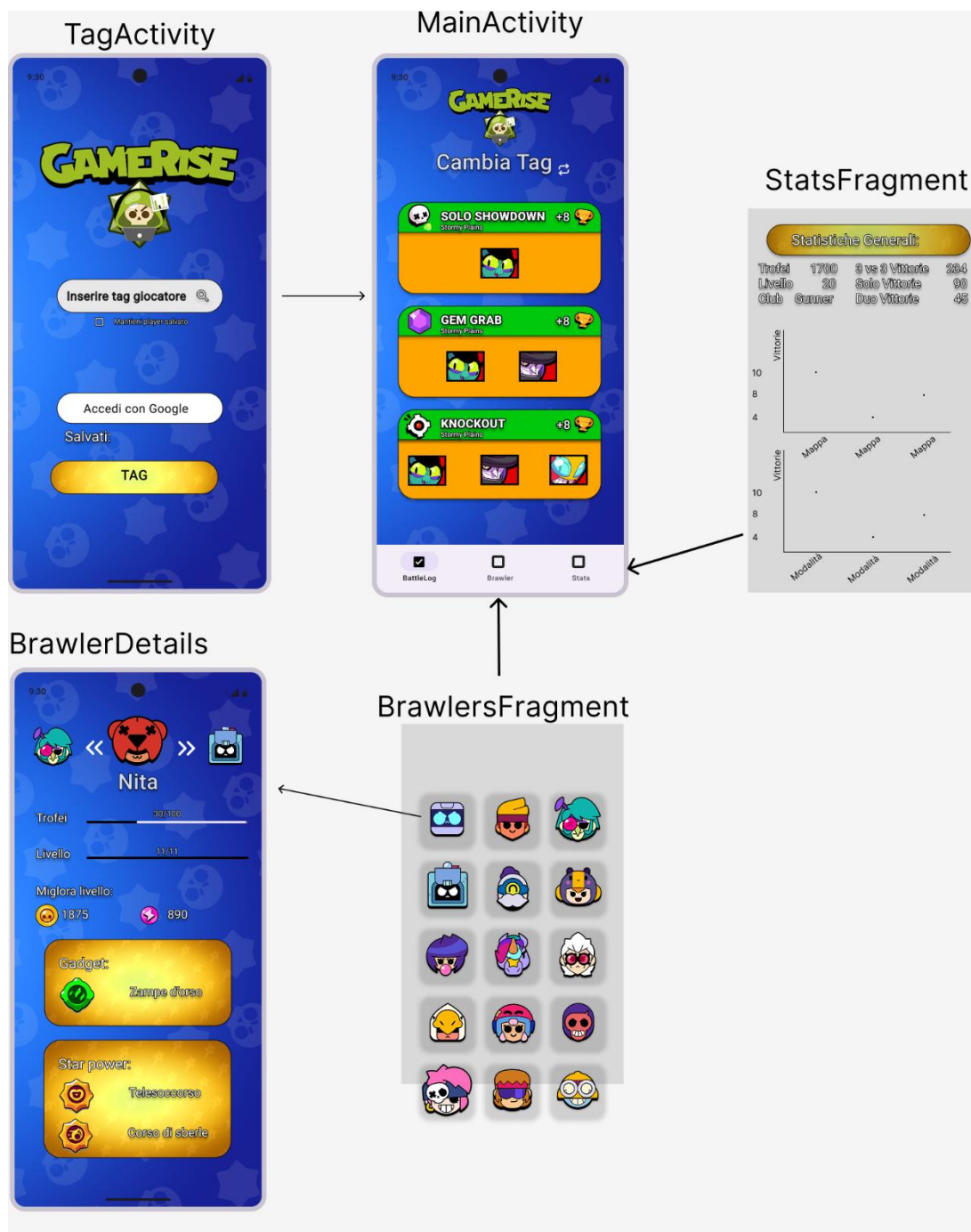


Figura 1: prototipo interfaccia

4 ARCHITETTURA

L'architettura segue lo standard **MVVM** (Model-View-ViewModel) e si basa sui seguenti component principali:

- **UI (User Interface):** gestisce l'input e l'output degli utenti e l'aggiornamento della visualizzazione, osservando i LiveData dal ViewModel.
- **ViewModel:** mantiene i dati (memorizzati in LiveData, elementi osservabili) che popolano la UI e ne gestisce la logica, notificando ogni cambiamento. Si interfaccia con il Repository per l'accesso ai dati.
- **Repository:** questo livello si occupa di recuperare i dati dallo strato sorgente ed esporli al resto dell'applicazione.
- **Data Source:** le classi di questo strato sono responsabili del recupero dei dati delle varie fonti (API, Firebase, Local Database), avvalendosi anche di librerie esterne come Retrofit e Room.

Per l'implementazione sono stati inoltre impiegati design pattern quali: Observer, Adapter, Factory e Singleton.

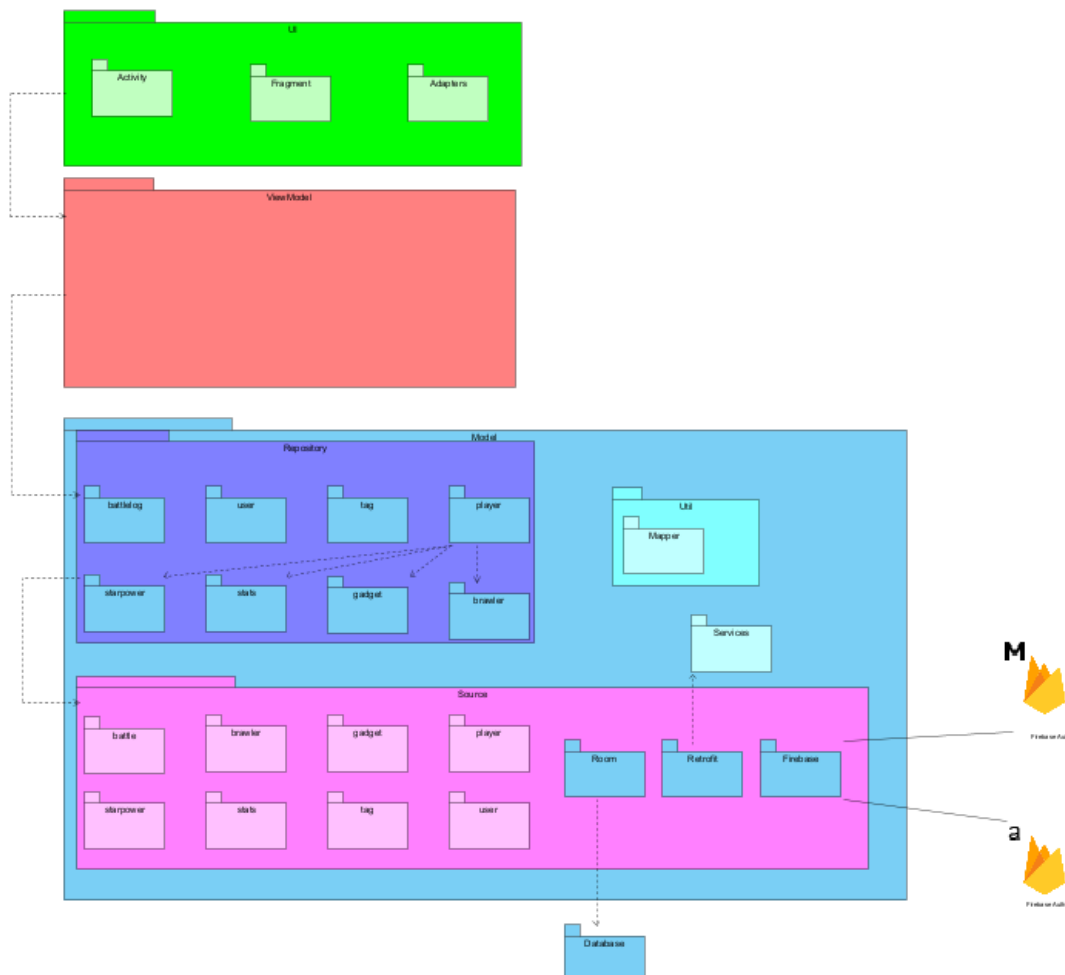


Figura 2: architettura logica

4.1 DATABASE

La classe GameRiseDatabase è un'implementazione di Room Database che gestisce l'archiviazione locale dei dati per l'app. Definisce un database contenente diverse entità (Stat, Tag, Battle, ecc.) e fornisce DAO per accedere ai dati. Utilizza un singleton per garantire un'unica istanza del database e un ExecutorService per eseguire operazioni di scrittura in background.

Per ogni entità del database è stato creato un DAO per gestire l'inserimento, la raccolta e il cancellamento dei dati. Inoltre, è stato implementato un PlayerTransitionDao che permette l'inserimento sequenziale di Brawler, StarPower e Gadget, grazie all'annotazione @Transaction, garantendo così l'atomicità delle operazioni e prevenendo eventuali problemi dovute alle chiamate asincrone all'API.

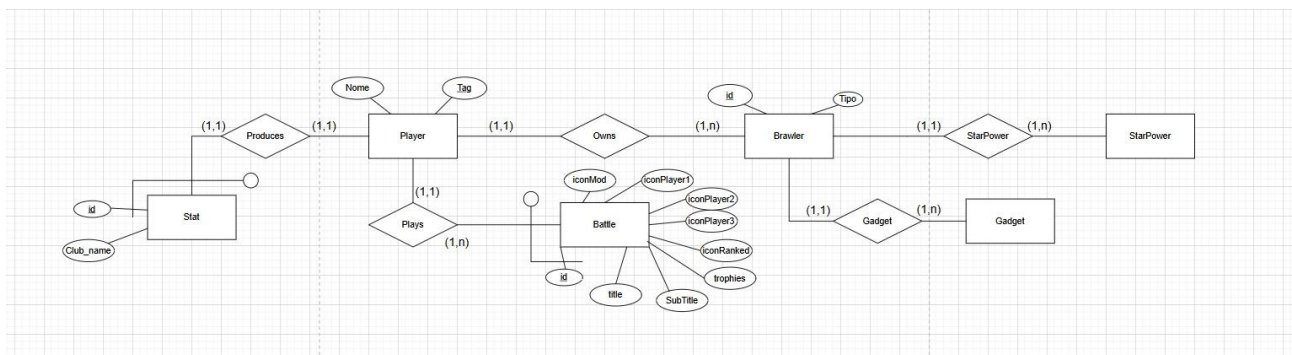


Figura 3: UML roomDatabase

4.2 MODEL

Rappresentano le strutture dati fondamentali utilizzate in diverse componenti dell'applicazione.

4.3 REPOSITORY

I repository gestiscono il recupero dei dati dal database locale o dall'API, decidendo quale utilizzare in base alla connessione e al tempo dell'ultimo aggiornamento. Utilizza `MultipleLiveData<Result>` per aggiornare l'UI con i dati o eventuali errori. Implementa le Callback, che gestisce il successo o il fallimento del recupero dati da entrambi le fonti. Questo approccio è stato utilizzato per tutte le entità, creando specifici Repository e Callback.

- **PlayerRepository**: funge da manager (repository multilivello) e astrae la logica di fetch API in modo da ridurre il numero di chiamate Api

4.4 SERVICE

Definisce le API REST dell'app utilizzando Retrofit per effettuare richieste HTTP GET con parametri dinamici ai servizi di gioco. Contiene diversi endpoint per ottenere informazioni su giocatori, battaglie, classifiche, ecc.

4.5 SOURCE

Definisce il recupero e l'inserimento dei dati da una fonte locale (database Room) e da una remota (API REST) attraverso classi astratte (BaseLocalDataSource e BaseRemoteDataSource).

Il LocalDataSource recupera i dati dal database usando i DAO e esegue operazioni di scrittura in background.

Il RemoteDataSource interroga un'API REST con Retrofit, converte la risposta e la passa al callback. In caso di errore, entrambi i data source notificano il problema tramite il callback.

4.6 OFFLINE-FIRST

L'applicazione permette di salvare il profilo di un giocatore di Brawl Stars in un Database locale, in modo che anche senza connessione ad Internet un utente possa vedere le statistiche dell'ultima volta che ha sincronizzato l'app

5 FUNZIONALITÀ

5.1 TAG

La schermata funge da punto centrale per la ricerca di un tag che può essere salvato o meno.

Offre inoltre la possibilità di effettuare il Login tramite Google. Se l'utente ha già effettuato l'accesso dal dispositivo in uso, l'applicazione eseguirà il login automatico utilizzando l'account precedentemente utilizzato.



Figura 4: TagActivy

5.2 BATTLE LOG

La schermata di Battle Log, che viene mostrata per prima all'avvio dell'applicazione, permette di visualizzare la lista delle battaglie disputate da quel giocatore. Per ogni battaglia vengono indicati il nome della battaglia, la mappa utilizzata, i trofei vinti e i brawler presenti nella squadra.

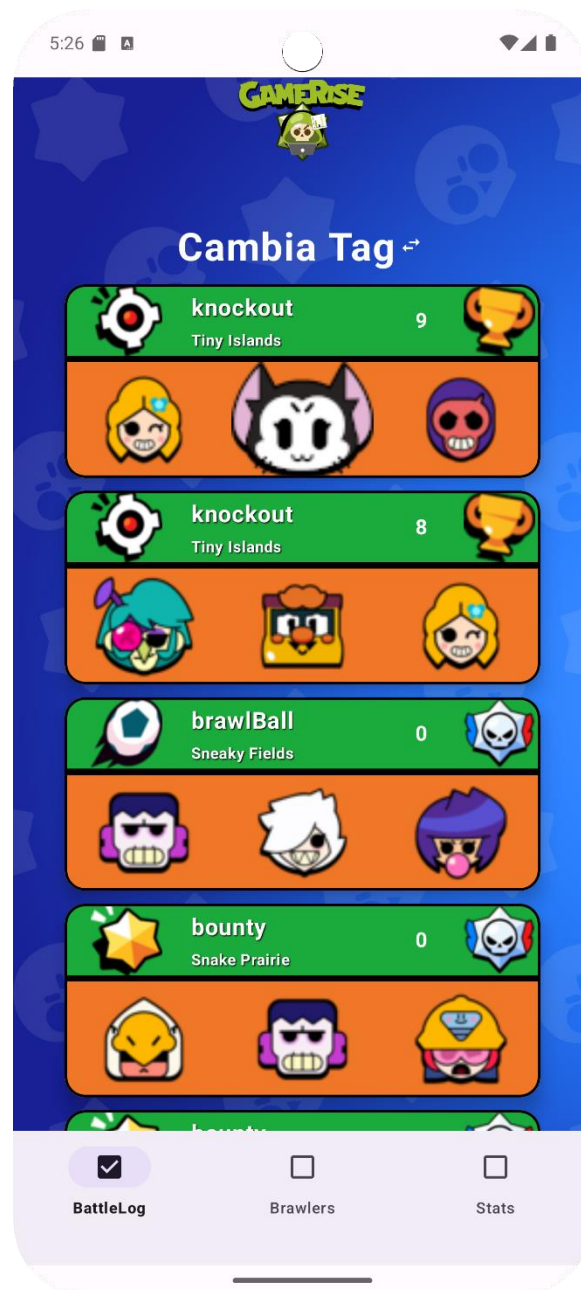


Figura 5: BattleLogFragment

5.3 BRAWLER

La schermata dei Brawler consente di visualizzare tutti i brawler posseduti dall'utente. Selezionando un brawler si passa alla visualizzazione dettagliata.

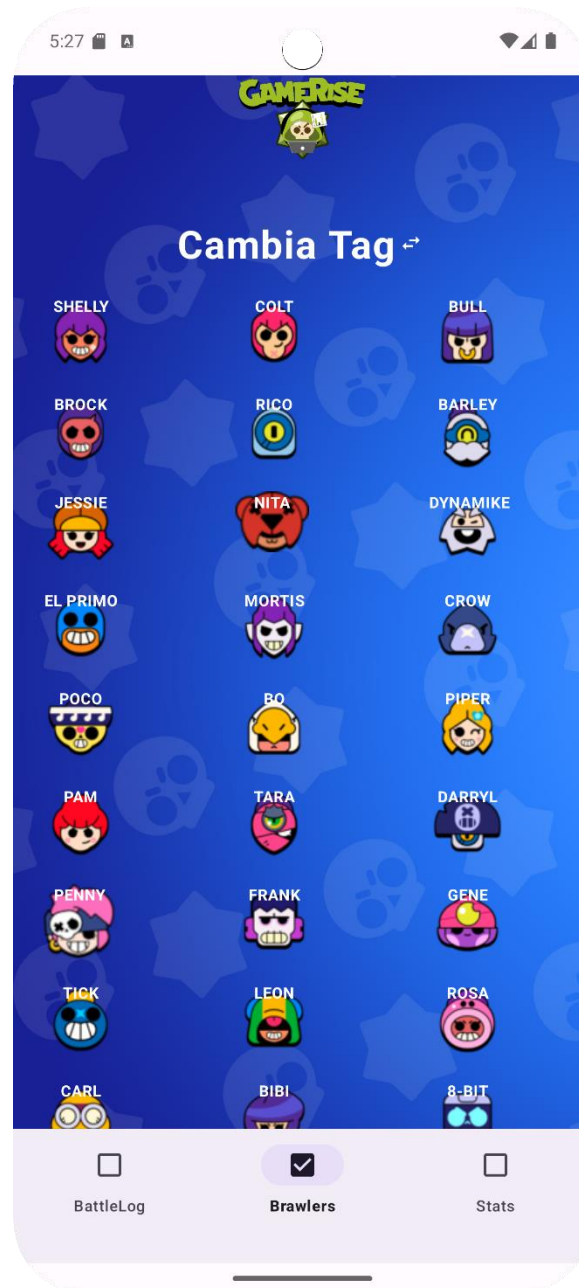


Figura 6: BrawlersFragment

5.4 STATS

La schermata delle statistiche permette di visualizzare i seguenti dati:

- Il totale dei trofei posseduti dall'utente;
- Il livello raggiunto;
- L'appartenenza a un club;
- Il numero di vittorie ottenute in modalità Solo;
- Il numero di vittorie ottenute in modalità Duo;
- Il numero di vittorie ottenute in modalità 3 vs 3;
- Un grafo per visualizzare quante partite ha vinto in ciascuna modalità;
- Un grafo per visualizzare quante partite ha vinto in ciascuna mappa.

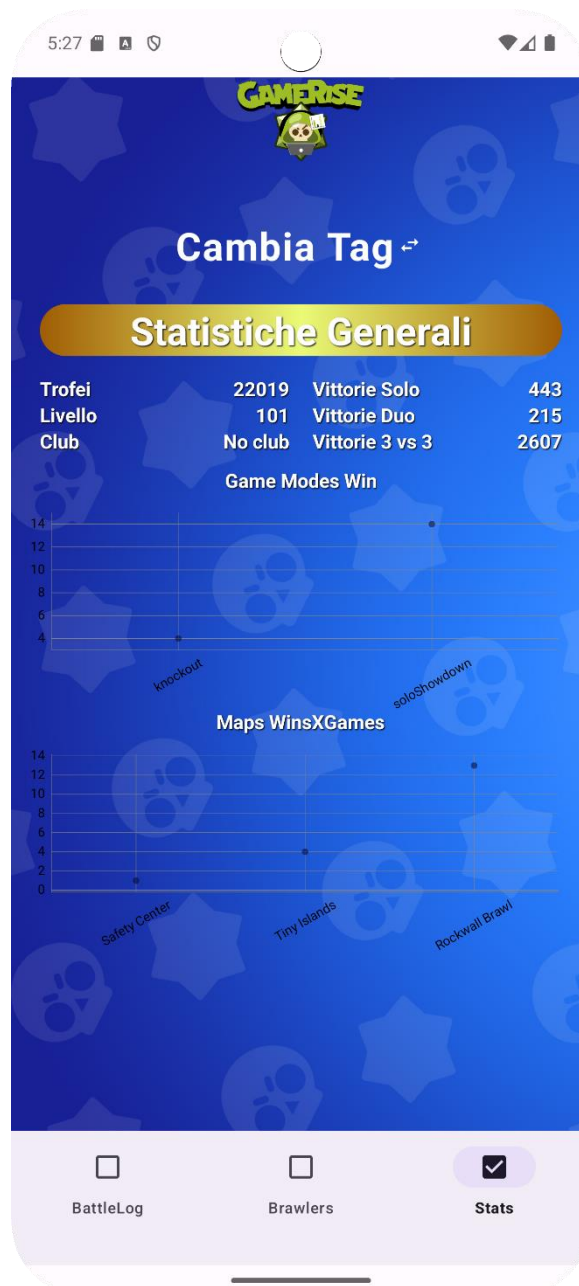


Figura 7: StatsFragment

5.5 BRAWLERDETAILS

Permette la visualizzazione di un brawler nel dettaglio, in particolare:

- Il numero di trofei ottenuti con quel brawler;
- Il livello raggiunto;
- La quantità di soldi e star point necessari per arrivare all'ultimo livello,
- I gadget posseduti;
- Le star power possedute.

È anche possibile passare al brawler precedente/successivo.



Figura 8: BrawlerDetailsActivity

6 SVILUPPI FUTURI

6.1 SUGGERIMENTI BRAWLER PER LE MAPPE

In futuro, verrà aggiunta una nuova pagina che, basandosi sulle statistiche calcolate, consiglierà il miglior brawler da utilizzare per ogni mappa.

6.2 ESPANSIONE A PIÙ GIOCHI MOBILE

L'idea iniziale del progetto era quella di creare un tracker multi-gioco, permettendo agli utenti di monitorare le proprie prestazioni su diversi titoli mobile. Questo obiettivo rimane nei piani futuri, con l'intenzione di estendere le funzionalità dell'app ad altri giochi multigiocatore.